Ders 9 Genişletilmiş Notu: Kaba Kuvvet, Kapsamlı Arama, Azalt ve Yönet, İkili Arama, Böl ve Yönet ve Master Teoremi

Ders 9 Genişletilmiş Notu: Kaba Kuvvet, Kapsamlı Arama, Azalt ve Yönet, İkili Arama, Böl ve Yönet ve Master Teoremi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Algoritma Geliştirme Yaklaşımlarına Giriş
- 2. Kaba Kuvvet (Brute Force)
- 3. Kapsamlı Arama (Exhaustive Search)
- 4. Azalt ve Yönet (Decrease and Conquer)
- 5. İkili Arama (Binary Search)
- 6. Böl ve Yönet (Divide and Conquer)
- 7. Master Teoremi
- 8. Merge Sort ile Quick Sort Karşılaştırması ve Kararlılık
- Örnek Sorular

1. Algoritma Geliştirme Yaklaşımlarına Giriş

Hoca bu haftayla birlikte yeni birkaç konuya geçileceğini söyledi: kaba kuvvet (brute force) yaklaşımı, kapsamlı arama (exhaustive search), azalt ve yönet (decrease and conquer) ve böl ve yönet (divide and conquer). Bunların hepsi algoritma geliştirme teknikleridir. Bu derste her biri için örnekler yapıldı.

Dersin başında da konuşulduğu gibi, problemlere farklı algoritmik yaklaşımlarla çözüm üretilebilir. Hoca bunlara örnek olarak dinamik programlama (dynamic programming), dal ve sınır (branch and bound) tekniğini ve böl ve yönet yaklaşımını saydı. Bu yaklaşımların arasında kaba kuvvet adı verilen bir kavram vardır. Hemen arkasından da kapsamlı arama gelir.

2. Kaba Kuvvet (Brute Force)

2.1 Tanım

Hoca “brute force nedir?” diye sordu. Bir öğrenci, bir şifreyi ya da bir olayı çözmek için seçenekleri sırasıyla denemek olduğunu söyledi. Hoca bunun genel olarak doğru olduğunu, ama tanımın biraz daha geniş olduğunu belirtti:

Kaba kuvvet, bir problemi doğrudan çözmek için, üzerinde herhangi bir optimizasyon yapmadan uygulanan en basit yönteme verilen addır. Problem çoğunlukla doğrudan tanımından yola çıkılarak çözülür.

Şifre çözme örneğinde ilk akla gelen çözüm, şifreyi bulmak için olası bütün seçenekleri tek tek denemektir. Bu bir kaba kuvvet yaklaşımıdır. Hoca bunun biraz sonra görülecek başka bir adı olduğunu da söyledi: kapsamlı arama. Çünkü şifreyi bulmak için olası bütün seçenekler denenmektedir.

Kaba kuvvet yaklaşımı pek çok problemin çözümünde çok yaygın biçimde kullanılır. Dezavantajı şudur: Her zaman geçerli bir çözüm verir, ama bu çözüm optimize edilmiş olmayabilir. Buna karşılık bazen diğer algoritmik yaklaşımlar yerine kaba kuvveti kullanmak daha akılcıdır. Çünkü diğer yöntemlerin de kendi içinde dezavantajları vardır. Hoca bunun bir örneğini dersin ilerleyen kısmında, a^n hesabında gösterdi (Bölüm 4.4).

2.2 a^n Hesaplama

İlk örnek a^n değerinin hesaplanmasıydı. Kaba kuvvet yaklaşımı, a'yı n kez kendisiyle çarpmaktır:

$$a^n = \underbrace{a \cdot a \cdots a}_{n \text{ tane}}$$

Bu, temel matematiksel işlemi yaparken kendiliğinden uyguladığımız yaklaşımdır. Hoca, a^n hesaplanırken yalnızca art arda çarpma yapılmadığını, farklı bakış açılarının da olduğunu söyledi. Bunlar yeni algoritma tekniğine (azalt ve yönet) geçildiğinde değerlendirildi.

2.3 En Büyük Ortak Bölen (EBOB)

İkinci örnek, iki sayının, m ile n'nin en büyük ortak bölenini (greatest common divisor, gcd) bulmaktı. Hoca kaba kuvvet çözümünü sordu.

Bir öğrenci önce mod alıp sayıları birbirine atayarak ilerleyen yöntemi anlattı: a mod b hesaplanır, değerler yer değiştirilir ve böyle devam edilir. Hoca bundan farklı, kaba kuvvet olan bir yaklaşım istediğini söyledi. Başka bir öğrenci, sayıların bütün çarpanlarını bulup ortak olanları işaretlemeyi, sonra ortakların en büyüğünü almayı önerdi.

Hoca bu fikri şöyle toparladı: Sayılar tek tek denenir, iki sayının da ona bölünüp bölünmediğine bakılır. İki sayıyı da bölen sayıların en büyüğü bulunur. Böylece bütün seçenekler denenmiş olur. Kaba kuvvette ilk akla gelen yaklaşım, iki sayının ikiden itibaren sayılara bölünebilirliğine bakmak ve bölünebilirliğin en son kesiştiği noktayı bulmaktır.

> **Ek bilgi (derste anlatılmadı):** Öğrencinin ilk söylediği, mod alıp yer değiştiren yöntem Öklid algoritmasıdır. Kaba kuvvet değildir. Hoca bu yöntemi dersin ilerleyen kısmında değişken miktarda azaltma örneği olarak ele aldı (Bölüm 4.5). Kaba kuvvetin yaygın bir yazımı da $t = \min(m, n)$'den başlayıp iki sayıyı da bölen ilk t'ye kadar t'yi birer azaltmaktır (ardışık tam sayı kontrolü). Bulunan ilk t doğrudan EBOB olur.

2.4 Seçmeli Sıralama ve Kabarcık Sıralama

Hoca sıralama algoritmaları arasında kaba kuvvete uygun ilk algoritmaları sordu. Öğrenciler seçmeli sıralama (selection sort) ve kabarcık sıralama (bubble sort) dedi. Hoca bunların çok güzel iki örnek olduğunu söyledi.

Soru: Bu iki algoritmaya neden kaba kuvvet çözümü denir? **Cevap:** Bir öğrenci, akla n^2 geldiğini, elimizdeki bütün elemanları her eleman için sürekli taramak zorunda kaldığımızı söyledi. Hoca bunu onayladı ve iki gerekçe verdi:

- İki algoritmanın da en kötü durum karmaşıklığı n^2'dir.
- Algoritma, elimizdeki dizilimin iyi ya da kötü olmasına bakmadan her dizilim için aynı yaklaşımı uygular. Verinin sıralı, sırasız ya da karışık verilmesi bakış açımızı hiç değiştirmez.

Seçmeli sıralamada bir eleman alınır, geri kalanlarla karşılaştırılır, sonra o turdaki en küçük (ya da en büyük) elemanla yeri değiştirilir. Temel amaç, bir turda en küçüğü bulup başa almak, sonra aynı işi kalan $n - 1$ eleman için tekrarlamaktır. Kabarcık sıralamada bu kez yan yana duran ikili sayılar karşılaştırılır. Küçük olan başa, büyük olan sona itilecek şekilde her turda aynı iş yapılır. Bu, uygulanabilecek en basit yaklaşımlardan biridir.

Hoca bu algoritmaların karmaşıklık analizini derste yapmadı. Rahatça yapılabileceğini ve öğrencilerin nasıl yapıldığına bir göz atması gerektiğini söyledi.

2.5 Sıralı Arama

Arama algoritmaları içinde kaba kuvvet olarak adlandırılabilecek olan, doğrusal arama (linear search), diğer adıyla sıralı aramadır (sequential search). Yapılan iş, bütün elemanlara tek tek bakmaktır. Bunun sıralı dizi ve sırasız dizi için farklı sürümleri olabilir. Hoca arama algoritmalarının sıralı aramadan çok daha ileri gittiğini, karmaşıklığın oldukça azaltıldığını söyledi. Hash algoritmaları da dahil olmak üzere bunlar ileride görülecek ve aradaki fark bir kez daha uygulamalı olarak görülecek.

2.6 Metin Eşleme (String Matching)

Hoca kaba kuvvet metin eşleme algoritmasının bakış açısını sordu. Bir öğrenci, aranan dizgiyi (string) metin üzerinde harf harf bulmaya çalıştığını, bulamadığı noktada bir harf ilerleyip yeniden aradığını söyledi. Hoca bunun güzel bir özet olduğunu belirtti.

Örnek olarak birinci dizgi (metin) A, B, C, D, E, F, G alındı. İkinci dizgi (desen, pattern) bu metnin içinde aranıyor. Küçük olanın büyüğün içinde arandığı varsayılır. Kaba kuvvette desen metnin başına hizalanır ve karakterler karşılaştırılır. Uyuşmazlık olunca desen bir kaydırılır:

- Metnin A'sı ile desenin ilk karakteri E karşılaştırılır. Tutmaz. Desen bir kaydırılır.
- B ile E karşılaştırılır. Tutmaz. Desen bir kaydırılır.
- C ile E karşılaştırılır. Böyle devam edilir.

  Ders sonrası not: Desen EFG alınırsa Python ile yapılan kaba kuvvet denemesinde eşleşme metnin 4 numaralı konumunda (0'dan sayarak, E harfinde) bulunur. Bunun için toplam 7 karakter karşılaştırması yapılır: ilk dört konumda birer uyuşmazlık, son konumda üç eşleşme. Desen FG alınırsa eşleşme 5 numaralı konumda yine 7 karşılaştırmayla bulunur.

Hoca bu yaklaşımın yalnız metin için değil, biyoinformatikte gen dizilimlerinin tutup tutmadığına bakılan araştırmalarda da çok sık kullanıldığını söyledi. Çok maliyetli bir iştir, çünkü bütün konumların tek tek denenmesi gerekir. Geçen yıl veri yapılarında görüldüğü gibi ve bu yıl ilerleyen haftalarda da görüleceği gibi, bir metin içinde başka bir deseni aramak için çok daha akıllıca yaklaşımlar vardır. Genel şartlarda onları kullanmak daha akıllıcadır.

2.7 En Yakın Nokta Çifti ve Dışbükey Zarf

Hoca kaba kuvvetle çözülebilen iki problemi daha başlık olarak yazdı: en yakın nokta çifti (closest pair) ve dışbükey zarf (convex hull). Dışbükey zarf, hocanın deyişiyle içbükey ve dışbükey şekillerde kullanılan algoritmadır.

En yakın nokta çifti probleminde veri setinde birbirine en yakın iki nokta aranır. İlk akla gelen temel yaklaşım şudur: Her noktanın diğer noktalara uzaklığı hesaplanır. Her hesapta bulunan değer o ana kadarki minimumla karşılaştırılır ve en küçüğü saklanır.

3. Kapsamlı Arama (Exhaustive Search)

3.1 Tanım

Hoca, verilen örneklerin dışında bir kapsamlı arama örneği istedi. Bir öğrenci "exhaustive" kelimesinin "zorlu" gibi bir anlam taşıdığını söyledi. Hoca buradan yola çıkarak çok yoğun, yorucu bir aramadan söz edildiğini belirtti. Başka bir öğrenci "bütün olasılıkların denenmesi" dedi.

Tanım şudur: Kombinatorik problemlerde bütün olasılıkların denenmesine kapsamlı arama denir.

3.2 Gezgin Satıcı Problemi (Traveling Salesman Problem)

İlk örnek meşhur gezgin satıcı problemidir (TSP). Bir öğrenci problemi şöyle hatırladı: Bir postacı ya da pazarlamacı şehirleri gezecek, her şehre yalnız bir kez uğrayacak ve sonra başladığı şehre geri dönecek. Başka bir öğrenci bunun en kısa yolu bulmak gibi bir şey olduğunu söyledi.

Hoca, en kısa yol bulma algoritmalarının farklı farklı amaçlar için kullanıldığını hatırlatarak tanımı netleştirdi.

Şehirler vardır. Bir pazarlamacı, postacı ya da günümüzde bir lojistik firması düşünülebilir. Dağıtılacak ürünler vardır ve her şehre bir ürün bırakılacağı varsayılır. Bir çıkış noktası vardır. Bu çıkış noktasından hangi rota takip edilerek her noktaya uğranabileceği hesaplanır.

Bu problem bütün noktalar arasında bir graf yapısı gerektirir. Bütün noktalar arasındaki kenarların (edge), yani yolların uzunlukları hesaplanır ve bunların olası kombinasyonlarına bakılır. Türkiye'de 81 il olduğu düşünülürse, bütün bu illerin kombinasyonlarını denemek çok çok yüksek bir miktar demektir. Bu nedenle kapsamlı arama algoritmalarında n belirli bir değerin üzerine çıkarsa problem kapsamlı aramayla çözülmez. Alternatif çözümlerin peşine düşülür. Hoca bu problemlere ileride daha ayrıntılı dönüleceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Başlangıç şehri sabitlenirse n şehirli bir TSP'de denenecek tur sayısı $(n-1)!$'dir. Kenar uzunlukları iki yönde aynıysa her tur ters yönüyle aynı uzunlukta olduğundan farklı tur sayısı $(n-1)!/2$'ye iner. Bu sayı faktöriyel olarak büyüdüğü için kapsamlı arama küçük n'ler dışında kullanılamaz.

3.3 Sırt Çantası Problemi (Knapsack Problem)

Hoca problemi bir hikâyeyle anlattı: Bir hırsız bir eve giriyor. Çuvalının alabileceği bir kapasite var. Evdeki eşyaların ağırlıkları ve değerleri biliniyor. Hırsız, bunları gözeterek çuvalına sığdırabileceği en yüksek toplam değere sahip hangi eşyaları seçmelidir?

Hoca bunun işin hikâyesi olduğunu, aynı senaryonun günlük hayattaki başka problemlere de uygulanabileceğini söyledi. Kısıtları (constraint) olan birçok maksimizasyon ve minimizasyon problemi bu yapıdadır.

Bu problem için henüz görülmemiş, çok daha etkili algoritmalar vardır. Kapsamlı aramada ise eşyaların bütün tekli, ikili, üçlü, ..., n'li kombinasyonları, yani bütün alt kümeler tek tek denenir. n eşya varsa alt küme sayısı 2^n'dir.

- 10 eşya varsa $2^{10} = 1024$ olasılık vardır. Bunların içinden kapasiteye sığan ve en yüksek değeri üreten olasılık bulunur. Hırsız o eşyaları alırsa hem çuvalına sığar hem de en yüksek değerli eşyaları almış olur.
- 10 eşyada sorun yoktur. Eşya sayısı 100'e çıkınca 2^{100} olasılık oluşur ve bu, işin rengini bir anda değiştirir.

Bu tür problemlere kombinatorik problemler denir. İçlerinde permütasyon ya da kombinasyon olabilir. Genelde NP-zor (NP-hard) problemler olarak karşımıza çıkarlar. Hoca bunların dersin son haftalarında, büyük olasılıkla son haftasında daha ayrıntılı görüleceğini söyledi.

> **Ders sonrası not:** Python ile $2^{10} = 1024$ doğrulandı. $2^{100} = 1\,267\,650\,600\,228\,229\,401\,496\,703\,205\,376$ yani yaklaşık $1{,}27 \times 10^{30}$.

4. Azalt ve Yönet (Decrease and Conquer)

4.1 Tanım

Hoca, kaba kuvvet ve kapsamlı arama için bu kadar örneğin yeterli olduğunu, dersin asıl konusunun azalt ve yönet ile böl ve yönet olduğunu söyledi.

Önce bir öğrenci böl ve yöneti tarif etti: Elimizdeki problemi farklı küçük problemlere bölüp hepsini ayrı ayrı çözmek, sonra yeniden birleştirmek. Hoca bunun güzel olduğunu söyleyip azalt ve yönette bunun nasıl değişeceğini sordu. Öğrenci, bakılan verinin hepsine bakmak yerine her seferinde azaltarak ilerlenebileceğini söyledi. Hoca "aynen öyle" dedi.

Azalt ve yönette problem, aynı problemin daha küçük bir örneğine (instance) indirgenir. Küçük problem çözülür, sonra bu çözüm genişletilerek asıl problemin çözümü elde edilir. Hoca bunu bir çizimle gösterdi: n boyutlu problem, daha küçük bir alt probleme (subproblem) indirilir. Alt problemin çözümü üretilir ve bu çözümden orijinal problemin çözümüne geçilir.

4.2 Üç Türü

Hoca azalt ve yönetin üç temel kullanım şeklini saydı:

- **Sabit miktarda azaltma (decrease by a constant):** Problem her adımda sabit bir değer kadar küçültülür. Bu değer çoğunlukla 1'dir.
- **Sabit oranda azaltma (decrease by a constant factor):** Problem her adımda sabit bir oranla küçültülür. Bu oran turlar arasında değişmez. Bilgisayar bilimlerinde çoğunlukla 2'dir.
- **Değişken miktarda azaltma (variable size decrease):** Her turda azaltılan miktar değişebilir.

  Ders sonrası not: Hoca sabit oranda azaltmayı anlatırken "her turda o değişkenin değeri kadar azaltıyorsunuz" dedi. Bu ifade sabit miktarda azaltmaya uyar. Sabit oranda azaltmada problem boyutu her turda o sabitten **çıkarılmaz**, o sabite **bölünür**. Örneğin oran 2 ise n boyutlu problem $n/2$ boyutlu probleme iner.

4.3 a^n ile Üç Yaklaşım

Hoca, kaba kuvvette görülen a^n problemine dönüp bunun azalt ve yönetle nasıl yazılacağını sordu.

Bir öğrenci $a^{n/2} \cdot a^{n/2}$ biçiminde, bölerek azaltmayı önerdi. Hoca bunun doğru olduğunu, ama o noktaya geçmeden önce daha basit bir yaklaşım istediğini söyledi. Başka bir öğrenci her turda üssü bir azaltmayı önerdi. Hoca bunu formül hâline getirdi.

Birer azaltma (decrease by one): Her seferinde üs bir azaltılır, yani $a^n = a^{n-1} \cdot a$ yazılır. Bu örnekte azaltma miktarı 1'dir. $f(n) = a^n$ dersek:

$$f(n) = \begin{cases} f(n-1) \cdot a & n > 0 \\ 1 & n = 0 \end{cases}$$

Hoca alt satırın koşulunu sordu. Bir öğrenci "n eşittir 0" dedi. Hoca, bunun geçerli olduğunu bilmekle birlikte matematiksel olarak ifade etmek gerektiğini söyledi. Azalt ve yönet ile böl ve yönet algoritmalarında özyinelemeli (recursive) bir çözüm üretmenin peşindeyiz. Burada problem özyinelemeli olarak formüle edildi ve durma noktası da belirlendi. $n = 0$ olduğu noktada 1 döner. Bu, başlangıç koşuludur (initial condition).

Sabit oranda azaltma (decrease by a constant factor): Hoca bir sonraki adımda bunu sordu. Bir öğrenci 1 yerine 2 çıkarılabileceğini söyledi. Hoca bu yaklaşımın tam doğru olmadığını belirtti. Başka bir öğrenci toparladı: Üssü her defasında 1 azaltmak yerine n'nin bölünebileceği bir tam sayıya bölmek. Hoca "aynen dediğin gibi" dedi ve özelleşmiş hâli yazdı:

$$a^n = \left(a^{n/2}\right)^2$$

Bu iki ifade birbirine denktir. Burada her seferinde n yerine $n/2$ hesaplanır. Hoca $n/3$ ya da $n/4$ da olabileceğini söyledi. Genelde bilgisayar bilimlerinde problemler ikiye bölünerek çözülür, böl ve yönette de bu görülecek. Ama istisnai durumlar (exceptional case) vardır. Problem 3'e, 4'e ya da genel olarak k'ya bölünerek de çözülebilir.

Ders sonrası not: Hoca burada "constant faktörümüz aslında $n/2$" dedi. Sabit oran $n/2$ değil, 2'dir. $n/2$ alt problemin boyutudur.

Soru: Bu yaklaşımla çözerken neye dikkat etmek gerekir, ne olursa sorun çıkar? **Cevap:** Bir öğrenci, n tekse ikiye bölmenin sorun çıkaracağını, bu yüzden bir a ile daha çarpmak gerektiğini söyledi. Hoca formülü birlikte yazdırdı. n'nin pozitif tam sayı olduğu kabul edilir:

$$a^n = \begin{cases} \left(a^{n/2}\right)^2 & n \text{ çift ve pozitifse} \\ \left(a^{(n-1)/2}\right)^2 \cdot a & n \text{ tekse} \\ 1 & n = 0 \text{ ise} \end{cases}$$

Böylece a^n üç yolla hesaplanmış oldu: kaba kuvvetle (art arda çarparak), özyinelemeli birer azaltmayla ve özyinelemeli sabit oranda azaltmayla.

4.4 Hangi Yöntem Daha Akıllıca?

Hoca üç yöntemden hangisinin daha akıllıca olduğunu sordu. Bir öğrenci yarıya azaltmanın daha verimli göründüğünü, çünkü ikili aramaya benzettiğini, ikiye böldüğü için süre açısından diğerlerinden çok daha hızlı gideceğini söyledi. Başka bir öğrenci de buna katıldı. Hoca karşı görüşte olan var mı diye sorduktan sonra kendi görüşünü anlattı:

Böl ve yönet ya da azalt ve yönet yaklaşımları her zaman en iyi sonucu vermez. Bu yöntemlerde bir fonksiyon çağrısı (function call) yapılır ve bunun bir maliyeti vardır. Özyinelemeli işlemlerde fonksiyon çağrısı ve geri dönüş (return) her zaman bir maliyet oluşturur. Hocaya göre burada, ikili arama mantığındakinin aksine bazı değerler geride bırakılmıyor, hepsi hesaplanıyor. Bu nedenle hoca burada kaba kuvveti kullanmanın daha akıllıca olduğunu söyledi. İşlemcide bu işleri paralel yapabilecek çok sayıda çekirdek (core) yoksa ya da çok işlemcili (multiprocessor) bir sistem kullanılmıyorsa, işi aynı işlemci üzerinde ikiye bölme mantığıyla özyinelemeli yapmak çok akıllıca olmaz. Bu işlemler ancak dağıtılırsa anlam ifade eder.

> **Hocanın vurgusu:** Böl ve yönet yaklaşımının problem çözmede her zaman daha iyi olduğunu söyleyemeyiz. Bazen, burada olduğu gibi a'ları art arda çarpan kaba kuvvet yaklaşımı daha verimli bir algoritma sunar.
>
> **Ders sonrası not:** Hoca, yarıya azaltmada "değerlerin hepsinin yine hesaplandığını" söyledi. Bu, tahtaya yazılan formül için doğru değildir. $\left(a^{n/2}\right)^2$ yazıldığında $a^{n/2}$ yalnızca bir kez hesaplanır ve sonuç kendisiyle çarpılır. Her adımda en fazla iki çarpma yapıldığı ve n her adımda yarıya indiği için çarpma sayısı $\Theta(\log n)$ olur. Kaba kuvvette ise $n-1$ çarpma gerekir (ilk çarpan a alınırsa). Python ile 3^{100} hesaplandı: yarıya azaltma 10 çarpmayla doğru sonucu verdi. Kaba kuvvet ve birer azaltma ise yaklaşık 100 çarpma yaptı. Hocanın uyarısı yalnızca $a^{n/2} \cdot a^{n/2}$ iki ayrı özyinelemeli çağrıyla hesaplanırsa geçerlidir. O durumda Python denemesinde 99 çarpma sayıldı, yani iş yine $\Theta(n)$ olur. Fonksiyon çağrısı maliyeti konusunda hoca haklıdır, ama yarıya azaltmanın asimptotik üstünlüğü bu maliyetten büyüktür.

4.5 Değişken Miktarda Azaltma: Öklid Algoritması

Hoca son olarak değişken miktarda azaltmaya örnek istedi ve öğrencilerin kaba kuvvet örnekleri listesine bakmalarını söyledi. Bir öğrenci, EBOB için yazılan Öklid algoritmasını söyledi. Ama neden bu yaklaşımın kullanılabildiğini açıklayamadı.

Hoca açıklamayı kendisi yaptı: Öklid algoritmasında her seferinde mod alınır:

$$\gcd(m, n) = \gcd(n,\ m \bmod n)$$

Mod alma işleminin sonucu, alınan moda, yani sayılara bağlı olarak her seferinde değişir. Bu yüzden problemde ilerleme miktarı ve hızı her adımda değişir. Özyinelemenin her seviyesinde bir alta inildiğinde, alınan moda bağlı olarak azaltma miktarı farklıdır. Bu nedenle bu yaklaşıma değişken miktarda azaltma denir. Hoca bunun başka örneklerinin de zamanı gelince görüleceğini, şimdilik genel kavramları bilmenin yeterli olduğunu söyledi.

> **Ders sonrası not:** Python ile $1 \le m, n < 80$ aralığındaki bütün sayı çiftlerinde kaba kuvvet EBOB ($\min(m,n)$'den aşağı denemek) ile Öklid algoritması aynı sonucu verdi.

4.6 İleriye Bağlantı: Topolojik Sıralama

Hoca, azalt ve yönet yaklaşımının ileride topolojik sıralamada (topological sorting) da kullanılacağını söyledi. Bu, birbirine bağlı işlerde kullanılır. Örneğin ön koşullu dersler vardır. Bir dersi almadan diğerini alamazsınız. Ders planını adım adım bu bağımlılıklara göre oluşturmaya topolojik sıralama denir.

5. İkili Arama (Binary Search)

5.1 Hangi Tür Azaltma?

Aradan sonra hoca azalt ve yönetin üç türünü hatırlattı: sabit miktarda azaltma, sabit oranda azaltma ve değişken miktarda azaltma. Ardından, çok sık kullanılan ve daha önce görülen ikili arama algoritmasına geçti. İkili arama, sıralı aramayı bir adım ileri taşıyan ve oldukça verimli (efficient) bir algoritmadır.

Soru: İkili arama bu üç yaklaşımdan hangisine girer? **Cevap:** Bir öğrenci sabit oranda azaltma dedi, çünkü eleman sayısı her seferinde ikiye bölünüyor. Hoca onayladı: Her seferinde bakılacak eleman sayısı $n/2$'ye iner. İkili aramada orta noktaya bakılır.

> **Hocanın vurgusu:** İkili aramada veri setinin sıralı olması önemlidir. Sırasız bir veri setinde ikili arama yapma şansımız yoktur.

5.2 Girdi, Çıktı ve Sözde Kod

Hoca algoritmanın ayrıntısına çok girmeden yalnız girdisini, çıktısını ve sözde kodunu verdi.

- **Girdi:** n elemanlı, sıralı bir $A[0..n-1]$ dizisi ve aranan K değeri. Dizinin hangi yönde sıralı olduğuna göre arama yönü değiştirilebilir.
- **Çıktı:** Bir öğrenci, aranan elemanın indisinin döndürülebileceğini, eleman yoksa -1 döndürülebileceğini söyledi. Hoca bunu kabul etti.

Sözde kod öğrencilerle birlikte yazıldı. Sol sınır için l kullanılır ve 0'dan başlar. Sağ sınır için r kullanılır ve $n-1$'den başlar.

```
BinarySearch(A[0..n-1], K)     // A sıralı
  l <- 0; r <- n-1
  while l <= r do
    m <- floor((l + r) / 2)
    if K = A[m] return m
    else if K < A[m] r <- m - 1
    else l <- m + 1
  return -1
```

Satır satır öğrencilerle kurulan kısımlar şunlardır:

- **while koşulu:** Bir öğrenci "right büyük eşittir left" dedi. Hoca bu şekilde yazılabileceğini söyledi.
- $K = A[m]$ **kolu:** Aranan eleman bulunmuştur. Bir öğrencinin önerisiyle `return m` yazıldı.
- $K < A[m]$ **kolu:** Bir öğrenci önce sağ sınıra K atanacağını, sonra m atanacağını söyledi. Hoca K'nın zaten aranan eleman olduğunu, m atamanın da tam doğru olmadığını belirtti. Başka bir öğrenci doğrusunu söyledi: $r \leftarrow m-1$. Gerekçe: K, $A[m]$'den küçük olduğu için m'yi yeniden almaya gerek yoktur, $m-1$'den başlamak mantıklıdır.
- **else kolu:** Tam tersidir.
- **Döngü sonu:** Eleman bulunamamışsa -1 döndürülür.

> **Ders sonrası not:** Derste else kolu için bir öğrenci "L'yi $m-1$ yapacağız" dedi ve bu sözlü olarak düzeltilmedi. Doğrusu $l \leftarrow m+1$'dir. $K > A[m]$ ise aranan eleman sağ yarıdadır. $r \leftarrow m-1$ yalnızca $K < A[m]$ kolundadır.

5.3 Yinelemeli mi, Özyinelemeli mi?

Soru: Bu sözde kod hangi yaklaşımla yazılmıştır, yinelemeli (iterative) mi, özyinelemeli (recursive) mi? **Cevap:** Bir öğrenci yinelemeli dedi. Hoca onayladı: Burada kendi kendini çağıran bir fonksiyon yoktur. Ama bu, ikili aramanın özyinelemeli bir çözümü olmadığı anlamına gelmez. İkili aramanın özyinelemeli bir sürümü de vardır ve bu gözden kaçırılmamalıdır.

5.4 En Kötü Durum Analizi

Hoca, asıl işin algoritmanın ayrıntısını konuşmak değil, bu algoritmik çözümün analizini yapmak olduğunu söyledi ve bir bağıntı yazdı:

$$C_w(n) = C_w(\lfloor n/2 \rfloor) + 1$$

Hoca bu bağıntının parçalarını tek tek sordu:

- w**:** Bir öğrenci en kötü durumu (worst case) gösterdiğini söyledi.
- $n/2$**:** Verinin her seferinde ikiye bölündüğünü, bunun teorik olarak ifade edildiğini gösterir.
- $+1$**:** Bir öğrenci bunun m için yapılan $+1$ ve -1 olduğunu söyledi. Hoca bunun doğru olmadığını belirtti. Başka bir öğrenci m'yi bulmak için yapılan atama işlemi olabileceğini söyledi. Bu da doğru değildi. Bir başka öğrenci "karşılaştırma" dedi ve doğru cevap buydu.
- C**:** Karşılaştırmanın (comparison) C'sidir.

Hoca hangi karşılaştırmanın temel işlem (basic operation) olduğunu da sordu. Bir öğrenci $K < A[m]$ karşılaştırmasını söyledi, çünkü en çok onunla karşılaşılacağını düşündü. Hoca, çok kritik olmamakla birlikte burada temel işlem olarak $K = A[m]$ karşılaştırmasının seçildiğini söyledi. Bu karşılaştırma her turda bir kez yapılır. $+1$ bundan gelir.

Son olarak hoca, şimdiye kadar yazılmayan şeyin ne olduğunu sordu. Bir öğrenci başlangıç (bitiş) koşulunu söyledi. Hoca bunun şöyle yazılabileceğini belirtti:

$$C_w(1) = 1$$

Hoca çözümün ayrıntısına girmedi, çünkü öğrenciler geriye doğru yerine koyma (backward substitution) yöntemini biliyordu. $n = 2^k$ alınıp bağıntı geriye doğru yerine koymayla çözülünce $k + 1$ elde edilir. Bu da $\log_2 n + 1$'dir. Ara adımlar şöyledir (ara adım eklendi):

$$C_w(2^k) = C_w(2^{k-1}) + 1 = C_w(2^{k-2}) + 2 = \cdots = C_w(2^0) + k = 1 + k$$

$k = \log_2 n$ olduğundan $C_w(n) = \log_2 n + 1$ bulunur. Hoca daha ayrıntılı gösterimin de olduğunu söyledi. Genel n için $\log_2 n$ aşağı yuvarlanır:

$$C_w(n) = \lfloor \log_2 n \rfloor + 1 = \lceil \log_2(n+1) \rceil \in \Theta(\log n)$$

> **Ders sonrası not:** Python ile $n = 1, \ldots, 1999$ için bağıntı, yukarıdaki sözde kodun gerçek en kötü durum karşılaştırma sayısı, $\lfloor \log_2 n \rfloor + 1$ ve $\lceil \log_2(n+1) \rceil$ dört değerinin de birbirine eşit olduğu doğrulandı. Örneğin $n = 1, 2, 7, 8, 100$ için sonuç sırasıyla 1, 2, 3, 4, 7'dir.

Hoca, şimdiye kadar özyinelemeli bağıntıları geriye doğru yerine koymayla çözdüklerini, biraz sonra Master teoremini göstereceğini söyledi. Master teoremiyle böl ve yönet ile azalt ve yönet problemleri geriye doğru yerine koyma yapmadan çözülebilir.

6. Böl ve Yönet (Divide and Conquer)

Hoca, böl ve yönet algoritmalarında genel olarak hangi adımlardan geçildiğini yazdı:

1. Problemin örnekleri (instance), mümkünse eşit büyüklükte, daha küçük örnek gruplarına bölünür. Mümkün olan en küçük parçalara ayrılır.
2. Bu küçük gruplar çözülür.
3. Küçük gruplara uygulanan çözümlerin sonuçlarını birleştirmeye ihtiyaç varsa, bunlar bir araya getirilerek orijinal problemin çözümü elde edilir. Hoca bazı algoritmalarda bu adıma mutlaka ihtiyaç olduğunu söyledi.

7. Master Teoremi

7.1 Genel Böl ve Yönet Bağıntısı

Hoca, böl ve yönet problemlerine özel olarak Master teoreminin kullanılacağını, bu teoreme göre böl ve yönet problemlerinin karmaşıklığının hesaplanabileceğini söyledi. Önce teoremi

yazdı. Algoritmadan bulunması gereken bağıntı, slaytta kırmızı kutu içinde yazılı olan genel böl ve yönet bağıntısıdır (general divide and conquer recurrence):

$$T(n) = a\,T(n/b) + f(n)$$

Teoremin durumları, $f(n) \in \Theta(n^d)$ ve $d \geq 0$ için şöyledir:

$$T(n) \in \begin{cases} \Theta(n^d) & a < b^d \\ \Theta(n^d \log n) & a = b^d \\ \Theta(n^{\log_b a}) & a > b^d \end{cases}$$

7.2 a, b ve n Ne Anlama Gelir?

Hoca parametrelerin anlamını sordu ve öğrencilerin ikili arama örneğinden yola çıkarak tahmin etmesini istedi. Bir öğrenci n'nin veri boyutu, b'nin kaç parçaya bölündüğü, a'nın da her parçanın karmaşıklık katsayısı olabileceğini tahmin etti. Hoca a'ya kadar olan kısmın doğru olduğunu söyledi:

- n**:** Örnek (instance) sayısı, yani girdi boyutu.
- b**:** Bölme sayısı. Problem kaça bölünerek ele alınıyor.

a için hoca bir ipucu verdi: İkili arama ile merge sort'taki yaklaşım aynı mı? İkili aramada her adımın sonunda veri setinin yalnızca bir kısmıyla ilgilenilir. Merge sort'ta neyle ilgilenilir? Bir öğrenci "alt parçaların sayısı mı?" dedi. Hoca bunun daha doğru ifade edilmesini istedi ve sonunda kendisi tanımladı:

- a**:** Bölünen parçalardan kaç tanesinin çözülmeye çalışıldığı, yani kaçının peşinden gidildiği.

Hoca öğrencilerle konuşarak iki örneği netleştirdi:

- **İkili arama:** Dizi ikiye bölünür, ama aynı anda iki tarafın peşinden gidilmez. Yalnızca bir taraf çözülür. Yani $a = 1$, $b = 2$.
- **Merge sort:** Dizi ikiye bölünür ve iki taraf da çözülür. Yani $a = 2$, $b = 2$.

Soru: a ile b'nin farkını tam anlayamadım. Bir daha anlatır mısınız? **Cevap:** Böl ve yönet problemlerinde uğraşılan örnek kümesi bölünerek ve parçalar çözülerek ilerlenir. İkili aramada da merge sort'ta da örnek kümesi ikiye ayrılır. b, örnek kümesinin kaça ayrılarak çözüldüğünü gösterir. Bu yüzden ikisinde de $n/2$ yazılır. a ise bu parçalardan kaç tanesinin çözülmeye devam edildiğini, kabaca kaçının peşinden gidildiğini gösterir. İkili aramada ikiye bölünmesine rağmen diğer grupla ilgilenilmez. Tek bir grubun peşinden gidilir, o grup yine ikiye bölünür ve yine tek grubun peşinden gidilir. Merge sort'ta ise ikiye bölünen iki taraf da bir eleman kalana kadar bölünmeye devam eder. Sonunda birer elemandan başlanarak sıralı biçimde birleştirilir ve yukarı doğru ilerlenir. İki tarafla da ilgilenildiği için $a = 2$ olur.

Soru: Bu durumda a asla b'den büyük olamaz, değil mi? **Cevap:** Hoca "sanırım evet, öyle gözüküyor" dedi.

> **Ders sonrası not:** Hocanın bu cevabı doğru değildir. a, b'den büyük olabilir. Bir problem b parçaya bölünüp birden fazla, birbiriyle örtüşen alt problem üzerinde çalışılabilir. Örneğin büyük olasılıkla ileriki derslerde görülecek olan Karatsuba çarpmasında $T(n) = 3T(n/2)+\Theta(n)$, Strassen matris çarpımında ise $T(n) = 7T(n/2)+\Theta(n^2)$'dir. Master teoreminin üçüncü durumu da tam olarak a'nın büyük olduğu durumları kapsar.

Soru: Böl ve yönette her adımda ikiye bölüp iki alt problem elde ediyoruz. O zaman a doğrudan 2 çıkmaz mı? **Cevap:** Hoca, öğrencinin söylediğinin b'ye karşılık geldiğini açıkladı. İkiye bölündüğü için $n/2$ yazılır. b, öğrencinin söylediği şeyin karşılığıdır.

7.3 $f(n)$ Ne Anlama Gelir?

Hoca $f(n)$'nin çok önemli bir kısım olduğunu söyleyip anlamını sordu. Bir öğrenci, $f(n)$'nin hem bölme hem de bölünenleri en sonunda bütün problem için birleştirme kısmındaki iş olduğunu söyledi. Hoca bunu şöyle tamamladı: $f(n)$, böldükten sonra çözümleri birleştirme sırasında harcanan zamanı, aynı zamanda yapılan işi gösterir.

Hoca merge sort'tan örnek istedi. Bir öğrenci şöyle anlattı:

- **Bölme (divide):** Alt dizileri bulmak için önceki dizinin ortası bulunur. Dizi yalnızca ortadan bölünür. Bölmenin karmaşıklığı $\Theta(1)$'dir.
- **Birleştirme (combine):** n elemanlı sıralı bir dizi yeniden oluşturulur. Bu $\Theta(n)$'dir.

Hoca bunun yanlış bir ifade olmadığını, ama herkesin anlayacağı dilden anlatmak gerektiğini, bunun da birazdan yapılacağını söyledi. Merge sort ve quick sort'un ayrıntılı analizi bir sonraki derse kaldı.

7.4 Örnek: $T(n) = 2T(n/2) + 1$

Hoca şu bağıntıyı yazdı ve bu bağıntıya sahip algoritmaların karmaşıklığını Master teoremiyle bulmak istedi:

$$T(n) = 2\,T(n/2) + 1$$

Çözüm öğrencilerle birlikte adım adım yapıldı:

1. a**:** Bağıntı genel böl ve yönet bağıntısıyla eşleştirilir. $a = 2$.
2. b**:** $b = 2$.
3. d**:** Burada birkaç yanlış tahminden sonra bir öğrenci "0" dedi. Gerekçe: $n^0 = 1$. Hoca açıkladı: $f(n)$ kısmında n, n^2 ya da $\log n$ gibi bir şey yoktur, sabit bir sayı vardır. Herhangi bir sayının sıfırıncı kuvveti 1'dir. Bu yüzden $d = 0$ olur.
4. **Durum seçimi:** Bir öğrenci $a = 2 > b^d = 2^0 = 1$ olduğunu söyledi. Üçüncü durum geçerlidir.
5. **Sonuç:**

$$T(n) \in \Theta\left(n^{\log_b a}\right) = \Theta\left(n^{\log_2 2}\right) = \Theta(n^1) = \Theta(n)$$

Soru: d'yi bir daha anlatır mısınız? **Cevap:** Hoca formülde $f(n) \in \Theta(n^d)$ olduğu bir senaryonun konuşulduğunu hatırlattı. Bu örnekte $f(n)$'nin karşılığı 1'dir. Burası n, n^2 ya da n^3 da olabilirdi. 1 olduğu için n'nin 1'e denk gelen kuvveti sıfırıncı kuvvettir. Bu yüzden $d = 0$ olur. Burası n olsaydı $d = 1$ olacaktı.

Bir öğrenci, a'yı bulurken b^d ile karşılaştırmaya erken geçmeye çalıştı ve "a, b^d'den küçük olmalı" dedi. Hoca, önce a, b ve d'nin üçünün de bilinmesi gerektiğini, d bilinmeden karşılaştırma yapılamayacağını hatırlattı.

> **Ders sonrası not:** Python ile $T(1) = 1$ alınıp $T(n) = 2T(n/2) + 1$ bağıntısı $n = 2^k$ değerleri için hesaplandı. Sonuç $T(n) = 2n - 1$ çıktı (örneğin $T(1024) = 2047$). Bu, $\Theta(n)$ sonucunu doğrular. Aynı teorem ikili arama bağıntısına ($a = 1$, $b = 2$, $d = 0$, $a = b^d$) uygulanırsa ikinci durumdan $\Theta(n^0 \log n) = \Theta(\log n)$ çıkar. Bu, Bölüm 5.4'teki sonuçla aynıdır.

Soru: Bu üç durumu nereden elde ettik? Üç durum var ve her birine "böyle olur" dedik. **Cevap:** Bu, Master teoremidir. Teoremin ispatı derste anlatılmıyor. Bu formüller çeşitli matematiksel çalışmalar sonucunda ortaya konmuştur. Derste önemli olan, böl ve yönet yaklaşımıyla çözülen bir problemin bağıntısı kurulduğunda bu üç durumun nasıl yorumlanacağıdır. Sonuç teoremde verilmiştir.

> **Ek bilgi (derste anlatılmadı):** Merge sort için $a = 2$, $b = 2$ ve birleştirme işi $\Theta(n)$ olduğundan $d = 1$'dir. $a = b^d = 2$ olduğu için ikinci durum geçerlidir ve $T(n) \in \Theta(n \log n)$ çıkar. Bu, merge sort'un aşağıda öğrencinin söylediği karmaşıklığıyla aynıdır.

8. Merge Sort ile Quick Sort Karşılaştırması ve Kararlılık

Hoca, sıralama ailesinde en çok öne çıkan iki algoritmanın merge sort ve quick sort olduğunu söyledi. Rakipleri vardır, ama bu ikisi çok sık kullanılır.

Soru: Merge sort'un karmaşıklığı nedir? **Cevap:** Bir öğrenci $\Theta(n \log n)$ dedi. Hoca hangi durum için olduğunu sordu. Öğrenci en iyi, ortalama ve en kötü durumun üçünde de $n \log n$ olduğunu söyledi.

Soru: Quick sort'un karmaşıklığı nedir? **Cevap:** Bir öğrenci en iyi durumda $n \log n$, en kötü durumda $O(n^2)$ dedi. Hoca ortalama durumu sordu. Öğrenci onun da $n \log n$ olduğunu söyledi. Hoca, ortalama durumda belirli bir katsayıyla çarpıldığını, ama katsayı düşürüldüğünde $n \log n$ olduğunu belirtti.

Algoritma	En iyi	Ortalama	En kötü
Merge sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Quick sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$

Bu tabloya göre merge sort daha avantajlı görünür. Hoca merge sort'un dezavantajını sordu. Bir öğrenci, ek yer kapladığını, alan karmaşıklığının (space complexity) n olduğunu söyledi. Hoca onayladı: Merge sort ek bir alana ihtiyaç duyar.

Hoca merge sort'un quick sort'a, aynı şekilde heap sort'a göre önemli bir avantajı daha olduğunu söyledi: kararlı (stable) olması.

Soru: Kararlı olmak ne demektir? **Cevap:** Bir öğrenci, dizideki aynı değere sahip elemanların yer değiştirmemesi olduğunu söyledi. Ardından "aynı büyüklüğe sahip" diye de ekledi ve örnek verdi: Dizide iki tane 2 varsa, soldakine "2-1", diğerine "2-2" denirse, sıralamadan sonra yerleri değişmez. Hoca öğrencinin ilk ifadesinin daha doğru olduğunu söyledi. "Aynı büyüklükte" yerine "aynı değere sahip" demek gerekir. Çünkü sıralanan şey illa sayısal bir değer olmak zorunda değildir. Metin ya da kelime de sıralanabilir. Sonuçta sayısal değere dönüştürülebilir, ama önemli olan aynı değere sahip olmalarıdır.

Hoca tanımı bir örnekle tamamladı: Dizide üç tane 2 olsun. Biri ikinci sırada, biri beşinci sırada, biri yedinci sırada. Kararlı bir sıralamada bu üç 2 arka arkaya geldiğinde, aralarındaki sıra korunur. Yani yine 2, 5, 7 sırasıyla yer alırlar, karışık biçimde değil. Heap sort ve quick sort'un böyle bir dezavantajı vardır, kararlı değildirler.

Hoca, aradan sonra merge sort ve quick sort'un analizinin yapılacağını, birbirlerine göre farklılıklarının konuşulacağını ve vakit kalırsa bir örnek yapılacağını söyledi.

Örnek Sorular

Bu derste ekran kaydı ya da kare yoktur. Aşağıdaki sorular hocanın derste sözlü olarak sorduğu sorulardır.

1. (derste sözlü, 00:08:31) a^n değerini hesaplamak için kaba kuvvet yaklaşımı nedir?

> **Çözüm:** a'yı n kez kendisiyle çarpmak: $a \cdot a \cdots a$.

2. (derste sözlü, 00:10:01) m ile n'nin en büyük ortak böleninin kaba kuvvet çözümü nedir?

> **Çözüm:** Sayılar tek tek denenir. İki sayıyı da bölen sayılar bulunur ve bunların en büyüğü alınır. Mod alıp yer değiştiren yöntem (Öklid) kaba kuvvet değildir.

3. (derste sözlü, 00:12:08) Seçmeli sıralama ve kabarcık sıralamaya neden kaba kuvvet çözümü denir?

> **Çözüm:** İkisinin de en kötü durum karmaşıklığı n^2'dir. Ayrıca girdi sıralı, sırasız ya da karışık olsun, her dizilime aynı yaklaşımı uygularlar.

4. (derste sözlü, 00:36:04) a^n'yi sabit oranda azaltma (decrease by a constant factor) ile nasıl yazarsınız? n tekse neye dikkat etmek gerekir?

Çözüm: n çiftse $a^n = (a^{n/2})^2$, n tekse $a^n = (a^{(n-1)/2})^2 \cdot a$, $n = 0$ ise $a^n = 1$. n tekken ikiye bölünemediği için fazladan bir a ile çarpmak gerekir.

5. (derste sözlü, 00:43:04) Değişken miktarda azaltmaya (variable size decrease) kaba kuvvet örneklerinden hangisi örnek verilebilir? Neden?

Çözüm: EBOB için Öklid algoritması: $\gcd(m, n) = \gcd(n, m \bmod n)$. Her adımdaki azaltma miktarı mod sonucuna bağlı olduğu için adımdan adıma değişir.

6. (derste sözlü, 00:55:47) İkili arama azalt ve yönetin hangi türüne girer?

Çözüm: Sabit oranda azaltmaya. Bakılan eleman sayısı her adımda yarıya iner.

7. (derste sözlü, 01:00:58) İkili aramada $K \neq A[m]$ ve $K < A[m]$ ise ne yapılır? Diğer durumda ne yapılır?

Çözüm: $K < A[m]$ ise $r \leftarrow m - 1$. Diğer durumda ($K > A[m]$) $l \leftarrow m + 1$. $A[m]$ zaten kontrol edildiği için m yeniden aralığa alınmaz.

8. (derste sözlü, 01:06:00) İkili aramanın en kötü durum bağıntısı $C_w(n) = C_w(\lfloor n/2 \rfloor) + 1$'de w, $n/2$, $+1$ ve C neyi ifade eder? Başlangıç koşulu nedir?

Çözüm: w en kötü durumu, $n/2$ problemin her adımda yarıya inmesini, C karşılaştırma (comparison) sayısını, $+1$ her turda bir kez yapılan $K = A[m]$ karşılaştırmasını gösterir. Başlangıç koşulu $C_w(1) = 1$'dir. Çözüm $C_w(n) = \lfloor \log_2 n \rfloor + 1 \in \Theta(\log n)$.

9. (derste sözlü, 01:18:52) Genel böl ve yönet bağıntısı $T(n) = aT(n/b) + f(n)$'de a, b, n ve $f(n)$ neyi gösterir? İkili arama ve merge sort için a ve b kaçtır?

Çözüm: n girdi boyutu, b problemin kaça bölündüğü, a bu parçalardan kaç tanesinin çözüldüğü, $f(n)$ bölme ve birleştirme için harcanan iştir. İkili aramada $a = 1$, $b = 2$. Merge sort'ta $a = 2$, $b = 2$.

10. (derste sözlü, 01:29:39) $T(n) = 2T(n/2) + 1$ bağıntısına sahip bir algoritmanın karmaşıklığını Master teoremiyle bulun.

Çözüm: $a = 2$, $b = 2$, $f(n) = 1 = n^0$ olduğundan $d = 0$. $a = 2 > b^d = 1$, üçüncü durum: $T(n) \in \Theta(n^{\log_2 2}) = \Theta(n)$.

11. (derste sözlü, 01:40:32) Merge sort'un quick sort'a göre dezavantajı ve avantajı nedir?

Çözüm: Dezavantajı ek bellek (alan) ihtiyacıdır. Avantajı her durumda $\Theta(n \log n)$ olması ve kararlı (stable) olmasıdır. Quick sort ve heap sort kararlı değildir.

Dersin Toparlaması

Bu derste önce kaba kuvvet yaklaşımı tanımlandı: Problemi tanımından doğrudan, optimizasyon yapmadan çözmek. a^n hesabı, EBOB, seçmeli ve kabarcık sıralama, sıralı arama, metin eşleme, en yakın nokta çifti ve dışbükey zarf bunun örnekleriydi. Kombinatorik problemlerde bütün olasılıkları denemek anlamına gelen kapsamlı arama, gezgin satıcı ve sırt çantası problemleriyle anlatıldı. Bu problemlerde çözüm sayısı çok hızlı büyür ve bu yüzden çoğu NP-zor problemdir.

Ardından azalt ve yönetin üç türü (a^n ve Öklid algoritması örnekleriyle) ve sabit oranda azaltmanın tipik örneği olan ikili arama görüldü. İkili aramanın en kötü durumu $\Theta(\log n)$ olarak bulundu. Son olarak böl ve yönetin adımları ve bu tür bağıntıları geriye doğru yerine koyma yapmadan çözmeyi sağlayan Master teoremi anlatıldı. Hoca, böl ve yönetin her zaman en iyi yaklaşım olmadığını da vurguladı. Ders, merge sort ile quick sort'un karşılaştırması ve kararlılık kavramıyla bitti. Bu iki algoritmanın analizi aradan sonraki uygulama oturumuna bırakıldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.